

Implémentation, ServeurMultiThreade (v2)

Fabrice.Kordon@lip6.fr



Objectif : gérer la terminaison...

2

Principe

- Les clients savent combien ils sont
- Les clients «connaissent» le serveur

• Donc...

- ▶ Le dernier client «tue» le serveur

À reprendre

- Le client
- Le serveur
- FilesRequêtes



La classe UnClient revisitée

3

```
class UnClient implements Runnable {
    private int monId;
    private FileRequetes f;

    private Random generateur = new Random();
    private final Object reveil = new Object(); // attente réponse

    private static int cpt = 1;
    private static final Object mutex = new Object();
    private static int clientsActifs = 0; // compter les clients
    private Thread serveur; // connaître le serveur

    UnClient (FileRequetes fr, Thread serv) {
        f = fr;
        serveur = serv;
        synchronized (mutex) {
            monId = cpt++;
            clientsActifs++;
        }
    }

    private void trace (String m) {...}

    public void requeteTraitee () {...}

    public String toString () {...}
}
```


La classe UnClient revisitée

3

```
public void run() {
    this.trace("initialisé");
    Requete r = new Requete(this, generateur.nextInt(1000));
    this.trace("j'envoie la requête " + r);
    f.insererRequete (r);
    this.trace("j'attends le serveur");
    try {
        synchronized (veille) {
            veille.wait();
        }
    }
    catch (InterruptedException e) {
        System.out.println ("UnClient "+monId+", cela doit jamais arriver, " + e);
    }
    this.trace("ma requête a été traitée");
    synchronized(mutex) { // gérer la terminaison
        clientsActifs--;
        if (clientsActifs == 0) {
            this.trace("étant le dernier, je tue le serveur");
            serveur.interrupt();
        }
    }
}
}
```


La classe LeServeur revisitée

4

```
class LeServeur implements Runnable {
    private FileRequetes f;

    LeServeur (FileRequetes fr) {
        f = fr;
    }

    private void trace (String m) {
        System.out.println ("Serveur : " + m);
        Thread.yield(); // rendre la commutation possible
    }

    public void run() {
        try {
            Requete r;
            this.trace("initialisé");
            while (true) {
                r = f.extraireRequete(); // potentiellement bloquant
                this.trace("je traite " + r);
                new Thread (new Servant(r)).start();
            }
        }
        catch (Exception e) { // Gérer la terminaison
            this.trace("il n'y a plus personne, adieu monde cruel");
        }
    }
}
```


Changement dans FileRequetes

5

```
public Requete extraireRequete () throws InterruptedException {
    Requete vret;
    l.lock();
    try {
        while (occupe == 0) {
            this.trace("file vide");
            lire.await(); // on propage InterruptedException
        }
        vret = tableau[derniere];
        occupe--;
        derniere++;
        if (derniere == tableau.length) {
            derniere = 0;
        }
        ecrire.signalAll();
    }
    finally {
        l.unlock();
    }
    return vret;
}
```


Exécution

6

```
$ java ServeurMultiThreade2 2 3
UnClient 1 : initialisé
UnClient 3 : initialisé
UnClient 2 : initialisé
UnClient 3 : j'envoie la requête (client 3, 128)
UnClient 2 : j'envoie la requête (client 2, 241)
UnClient 1 : j'envoie la requête (client 1, 548)
Serveur : initialisé
FileRequetes : --> [0, 0,0]
UnClient 3 : j'attends le serveur
FileRequetes : --> [1, 1,0]
UnClient 2 : j'attends le serveur
Serveur : je traite (client 3, 128)
FileRequetes : --> [1, 0,1]
UnClient 1 : j'attends le serveur
Serveur : je traite (client 2, 241)
Serveur : je traite (client 1, 548)
Servant (client 3, 128) : initialisé
Servant (client 2, 241) : initialisé
FileRequetes : file vide
Servant (client 1, 548) : initialisé
Servant (client 3, 128) : je prévient le client
UnClient 3 : ma requête a été traitée
Servant (client 2, 241) : je prévient le client
UnClient 2 : ma requête a été traitée
Servant (client 1, 548) : je prévient le client
UnClient 1 : ma requête a été traitée
UnClient 1 : étant le dernier, je tue le serveur
Serveur : il n'y a plus personne, adieu monde cruel
$
```